AEC1 - Introducción a Scala

Etiquetas

Introducción

Esta práctica supone la primera toma de contacto real con Scala. En mi caso vengo de haber usado mayoritariamente Java y C++ como lenguajes principales, así que algunos conceptos del este nuevo lenguaje han sido un cambio bastante grande, especialmente la parte funcional y el sistema de tipos.

He organizado la entrega en tres partes:

- **Parte 1**: un cheat sheet con los elementos del lenguaje que más me han llamado la atención, ordenados por tema. Lo he ido rellenando a medida que avanzaba en los ejemplos de la unidad.
- **Parte 2** (programa2.scala): Aquí he diseñado un programa relativamente simple que trabaja con notas de clase aplicando programación funcional. Lo elegí porque creo que tiene bastante sentido como caso de uso real y me permitía usar bastantes operaciones de colecciones sin que quedara forzado.
- **Parte 3** (programa2.scala): En este caso he diseñado un programa con case class Estudiante, generación automática de instancias, lectura de un fichero con preproceso, dos implementaciones de agrupación y escritura del resultado filtrado.

Para ejecutarlos debemos hacer `scala programa1.scala` y `scala programa2.scala` desde el directorio de la práctica. El segundo necesita estudiantes.txt en el mismo directorio, que también está incluido.

Parte 1 — Mi Cheat Sheet de Scala

En esta sección recojo los elementos del lenguaje que más me han costado o que me parecen más distintos respecto a lo que había visto antes. Lo voy completando conforme avanzo en la asignatura.

Variables

```
val x: Int = 10    // inmutable, no se puede reasignar (como final en Jav
a)
var y: Int = 20    // mutable
val z = 3.14       // Scala infiere el tipo, no hace falta escribirlo siem
pre
```

Lo más llamativo es el uso de val por defecto. Si algo no necesita cambiar, declararlo mutable solo añade una fuente de bugs.

> En Scala no hace falta el punto y coma al final de cada línea. El compilador lo infiere. Tener cuidado si partes una expresión en dos líneas porque puede interpretarla de forma inesperada.

Tipos básicos

Tipo	Java equiv.	Notas
`Int`	`int`	-
`Double`	`double`	-
`Boolean`	`boolean`	-
`String`	`String`	-
`Any`	`Object`	supertipo de absolutamente todo
`AnyVal`	tipos primitivos	supertipo de Int, Double, etc.
`AnyRef`	`Object`	supertipo de clases
`Unit`	`void`	cuando una función no devuelve nada
`Nothing`	—	subtipo de todo, se usa en situaciones de error o bucles infinitos

Objetos y companion objects

```
object HolaMundo {
  def main(args: Array[String]): Unit = {
    println("Hola mundo")
  }
}

// companion object: mismo nombre que la clase, en el mismo fichero
class Persona(val nombre: String, val edad: Int)
object Persona {
  def apply(nombre: String, edad: Int): Persona = new Persona(nombre, eda
d)
}

val p = Persona("Ana", 25)  // llama a apply, no hace falta 'new'
```

El object en Scala es un singleton, no hay que instanciarlo. Si tiene el mismo nombre que una clase es un companion object y puede acceder a sus miembros privados.

Funciones

```
def suma(a: Int, b: Int): Int = a + b

// valor por defecto en parámetros
def saludo(nombre: String, msg: String = "Hola"): String = s"$msg,$nombre"

// función anónima
val doble: Int => Int = x => x * 2

// orden superior: recibe una función como parámetro
def aplicar(f: Int => Int, x: Int): Int = f(x)
aplicar(doble, 5)   // 10

// currificación (aplicación parcial)
```

```
def multiplicar(a: Int)(b: Int): Int = a * b
val por3 = multiplicar(3) _
por3(7)   // 21
```

Case Classes

```
case class Punto(x: Double, y: Double)

val p1 = Punto(1.0, 2.0)   // sin 'new'
val p2 = p1.copy(y = 5.0)  // copia con un campo cambiado
p1 == Punto(1.0, 2.0)      // true, equals estructural automático

p1 match {
  case Punto(0, 0) => println("origen")
  case Punto(x, y) => println(s"$x,$y")
}
```

Se generan automáticamente: toString, equals, hashCode, copy, apply y unapply. En Java esto serían decenas de líneas de boilerplate.

> El copy es especialmente útil: en lugar de mutar un objeto, creas uno nuevo con solo el campo que cambia. Es la forma idiomática de “modificar” datos inmutables en Scala.

Pattern Matching

```
val x: Any = 42
x match {
  case i: Int if i > 0 => println(s"positivo:$i")
  case s: String       => println(s"string:$s")
  case _               => println("otro")
}

// también con listas
List(1, 2, 3) match {
  case Nil    => "vacía"
  case h :: t => s"cabeza$h"
}
```

Mucho más potente que el switch de Java. Los guards (if dentro del case) permiten añadir condiciones adicionales sin anidar.

Option

```
val presente: Option[Int] = Some(42)
val ausente:  Option[Int] = None

presente.getOrElse(0)    // 42
```

```
ausente.getOrElse(0)     // 0
presente.map(_ * 2)      // Some(84)

presente match {
  case Some(v) => println(v)
  case None    => println("no hay nada")
}
```

Sustituye a nuestro querido null de Java. Obliga a pensar en el caso de ausencia de valor en el momento de escribir el código, no cuando ya ha petado en producción. Al principio cuesta un poco pero luego se agradece bastante.

Traits

```
trait Volador {
  def volar(): String = "volando"   // implementación por defecto
  def aterrizar(): String           // abstracto
}

class Pajaro extends Volador {
  def aterrizar(): String = "aterrizando"
}
```

Como interfaces de Java pero pueden tener implementación. Una clase puede mezclar varios con with.

Generics

```
class Caja[A](val contenido: A) {
  def obtener: A = contenido
}

def primero[A](lista: List[A]): Option[A] = lista.headOption

// upper bound: A debe extender Comparable
def maximo[A <: Comparable[A]](a: A, b: A): A =
  if (a.compareTo(b) > 0) a else b
```

Programación funcional y colecciones de orden superior

```
val nums = List(1, 2, 3, 4, 5)

nums.map(_ * 2)              // List(2, 4, 6, 8, 10)
nums.filter(_ % 2 == 0)      // List(2, 4)
nums.foldLeft(0)(_ + _)      // 15
nums.reduce(_ + _)           // 15
nums.flatMap(x => List(x, x * 10))  // List(1, 10, 2, 20, ...)
```

```
nums.groupBy(_ % 2 == 0)     // Map(true -> List(2,4), false -> List(1,3,
5))
nums.sortBy(-_)              // List(5, 4, 3, 2, 1)

val (pares, impares) = nums.partition(_ % 2 == 0)
```

- foldLeft: Recorre una lista para combinar todos sus elementos en un único resultado final (como una suma o un objeto), usando un valor inicial.
- flatMap: Transforma cada elemento en una nueva colección y luego las fusiona todas en una sola lista plana, eliminando los niveles extra.

> Un detalle importante: reduce y foldLeft no son equivalentes. reduce revienta si la lista está vacía porque no tiene valor inicial al que volver; foldLeft no porque se lo pasas tú. En la práctica, si no puedes garantizar que la lista tiene al menos un elemento, usa foldLeft.

For Comprehension

```
val resultado = for {
  x <- List(1, 2, 3)
  y <- List(10, 20)
  if x != 2
} yield x * y

// con Option
for {
  a <- Some(3)
  b <- Some(4)
} yield a + b   // Some(7)
```

Sirve para encadenar flatMap, map y filter. En lugar de escribir un código lleno de paréntesis y flechas anidadas, escribes una lista vertical que parece lenguaje humano.

- Queda mucho más legible cuando hay varias transformaciones encadenadas.
- También funciona con Option, Try y cualquier tipo que implemente flatMap, no solo con listas.

Try y manejo de errores

```
import scala.util.{Try, Success, Failure}

// estilo clásico
try {
  "abc".toInt
} catch {
  case e: NumberFormatException => println(e.getMessage)
} finally {
  println("siempre se ejecuta")
}
```

```
// estilo funcional, más cómodo para encadenar
Try("42".toInt) match {
  case Success(v) => println(v)
  case Failure(e) => println(e.getMessage)
}

Try("abc".toInt).getOrElse(-1)  // -1
```

Colecciones principales

```
val lista  = List(1, 2, 3)         // inmutable, enlazada
val v      = Vector(1, 2, 3)       // inmutable, acceso aleatorio eficient
e
val mapa   = Map("a" -> 1, "b" -> 2)
val conj   = Set(1, 2, 3, 2)       // Set(1, 2, 3)

// mutable si hace falta
import scala.collection.mutable
val buf = mutable.ListBuffer[Int](1, 2, 3)
buf += 4
```

Ficheros

```
import scala.io.Source
import java.io.{File, PrintWriter}

val lineas = Source.fromFile("datos.txt").getLines().toList

val pw = new PrintWriter(new File("salida.txt"))
pw.println("linea 1")
pw.close()
```

Interpolación de strings

```
val nombre = "Scala"
println(s"Hola$nombre,${1 + 1} mas")   // s para expresiones
println(f"Pi:$3.14159%.2f")              // f para formato numérico
println(raw"sin escapar \n esto")         // raw para no interpretar escap
es
```

Scala vs Java (resumen rápido)

Parte 2 — Programa Scala I

Qué hace

El programa trabaja con notas de una clase ficticia. Lo elegí como temática porque tiene sentido como caso real y me permitía usar bastantes operaciones de colecciones de forma natural, sin que el código pareciera un ejercicio de laboratorio.

El flujo es: tengo una lista de 10 notas, las filtro, les aplico transformaciones, las agrupo por calificación, y al final demuestro el uso del tipo Any con una lista que mezcla enteros, strings, doubles, booleanos y listas.

Explicación del código

```
val notas: List[Double] = List(8.5, 7.0, 9.2, 6.3, 5.8, 8.1, 7.7, 4.5, 6.
0, 9.8)
```

El punto de partida del programa. Declaro la lista con val porque no necesito reasignarla en ningún momento; es un dato fijo sobre el que voy a trabajar. En Scala la inmutabilidad por defecto es una buena práctica: si el compilador sabe que la variable no cambia, puede razonar mejor sobre el código y evita errores de concurrencia o modificaciones accidentales.

```
def calcularMedia(lista: List[Double]): Double =
  if (lista.isEmpty) 0.0 else lista.sum / lista.length
```

Función sencilla que calcula la media aritmética. El guard de isEmpty es importante: sin él, llamar a la función con una lista vacía haría una división 0/0 y devolvería NaN en lugar de lanzar una excepción o devolver 0. Funciones que controlan estos casos límite desde el principio son mucho más robustas.

```
def clasificarNota(nota: Double): String = nota match {
  case n if n >= 9.0             => "Sobresaliente"
  case n if n >= 7.0 && n < 9.0 => "Notable"
  case n if n >= 5.0 && n < 7.0 => "Aprobado"
  case _                         => "Suspenso"
}
```

Aquí uso pattern matching con guards, que son las condiciones if dentro de cada case. El compilador prueba los casos de arriba a abajo y se queda con el primero que se cumple. El case _ al final es el comodín: captura cualquier valor que no haya coincidido antes, en este caso notas negativas o lo que sea. Sin ese comodín el compilador avisaría de que el match no es exhaustivo.

```
def transformarNotas(notas: List[Double], f: Double => Double): List[Doubl
e] =
  notas.map(f)
```

Función de orden superior: recibe otra función como parámetro. El tipo Double ⇒ Double indica que f tiene que ser algo que dada una nota devuelva otra nota. Esto hace que transformarNotas sea genérica: puedo pasarle una subida del 10% con _ * 1.1, o redondear, o lo que quiera, sin reescribir la función.

```
val aprobados = notas.filter(_ >= 5.0)
```

filter devuelve una nueva lista con solo los elementos que cumplen el predicado. El guión bajo aquí es una lambda abreviada: _ representa cada elemento de la lista, así que _ >= 5.0 equivale a n ⇒ n >= 5.0. La lista notas no se toca; filter siempre crea una nueva.

```
val notasConCalificacion = notas.map(n => (n, clasificarNota(n)))
```

map transforma cada elemento y devuelve una lista nueva con los resultados. Aquí la función pasa de Double a una tupla de dos elementos: la nota original y su calificación como String. El resultado final es una List de tuplas (Double, String). Esto permite luego imprimirlas con foreach usando destructuring: case (nota, cal) ⇒ ...

```
val sumaTotal = notas.foldLeft(0.0)(_ + _)
```

foldLeft recorre la lista acumulando un resultado. Los dos parámetros son: el valor inicial del acumulador (0.0) y la función que combina acumulador con cada elemento (_ + _ es la abreviación de (acc, x) ⇒ acc + x). Es más seguro que reduce porque si la lista está vacía simplemente devuelve el valor inicial, no lanza excepción.

```
val agrupadas = notas.groupBy(clasificarNota)
```

groupBy recorre la lista una sola vez y construye un Map: aplica clasificarNota a cada nota para obtener la clave, y agrupa todas las notas que comparten clave en la misma lista. El resultado es Map[String, List[Double]]. A la hora de imprimirlo, llamo a toSeq.sortBy para ordenar las claves, porque un Map en Scala no garantiza ningún orden de iteración.

```
val elementosMixtos: List[Any] = List(42, "Scala", 3.14, true, List(1, 2,
3))
elementosMixtos.foreach(e => println(describir(e)))
```

Any es el supertipo de absolutamente todo en Scala, tanto tipos primitivos como objetos. Eso permite meter enteros, strings, doubles, booleanos y listas en la misma colección. La función describir hace pattern matching sobre el tipo concreto en tiempo de ejecución para saber qué es cada elemento y cómo mostrarlo.

Parte 3 — Programa Scala II

Qué hace

Modela estudiantes universitarios. La idea es demostrar la case class de forma completa: no solo crearla, sino usarla en lectura de fichero, agrupación, filtrado y escritura, que es lo que tiene sentido en la práctica real.

El flujo del programa es:

- Se generan 10 instancias automáticamente con valores consecutivos
- Se leen más instancias del fichero `estudiantes.txt`, que tiene comentarios y líneas vacías como preproceso

- Se combinan ambas listas
- Se agrupan por curso con dos implementaciones (imperativa y funcional) para comparar
- Se filtran los que tienen nota >= 7.0 y edad <= 22
- El resultado se escribe en `estudiantes_filtrados.txt` con el mismo formato

Explicación del código

```
case class Estudiante(id: Int, nombre: String, edad: Int, nota: Double, cu
rso: String)
```

Define la estructura de datos central del programa. Al declararla como case class, el compilador genera automáticamente varios métodos: apply (para crear instancias sin new), equals y hashCode basados en el valor de los campos (no en la referencia de memoria como en Java), toString con formato legible, y copy para crear una copia modificando solo algunos campos. En la práctica esto ahorra bastante código y evita bugs típicos de Java donde comparar dos objetos con == compara referencias en lugar de contenido.

```
val estudiantesGenerados: List[Estudiante] = (1 to 10).map { i =>
  Estudiante(
    id     = i,
    nombre = s"Estudiante_$i",
    edad   = 18 + i,
    nota   = Math.min(4.5 + i * 0.55, 10.0),
    curso  = cursos((i - 1) % cursos.length)
  )
}.toList
```

(1 to 10) crea un Range del 1 al 10 inclusive. map recorre ese rango y transforma cada valor i en un Estudiante, con edad, nota y curso calculados a partir de i para que sean distintos entre sí. El módulo (i - 1) % cursos.length hace que el curso rote cíclicamente: el primero coge cursos(0), el segundo cursos(1), y cuando llega al final vuelve al principio. Al final se necesita el toList porque map sobre un Range devuelve IndexedSeq, y el resto del programa trabaja con List.

```
def parsearLinea(linea: String, separador: String): Option[Estudiante] = {
  val partes = linea.split(separador)
  if (partes.length == 5)
    Try(Estudiante(partes(0).trim.toInt, ...)).toOption
  else
    None
}
```

Convierte una línea de texto en un Estudiante, si es posible. Primero divide la línea por el separador y comprueba que tiene exactamente 5 campos. Luego intenta construir el Estudiante convirtiendo los tipos (por ejemplo partes(0).toInt para el id). El Try envuelve esa construcción por si alguna conversión falla, por ejemplo si la nota tiene un formato incorrecto. toOption convierte el resultado: si fue bien devuelve Some(estudiante), si petó devuelve None. Así el error no interrumpe el programa.

```
def agruparPorCursoImperativo(es: List[Estudiante]): Map[String, List[Estu
diante]] = {
  var resultado = Map[String, List[Estudiante]]()
  for (e <- es) {
    val listaActual = resultado.getOrElse(e.curso, List())
    resultado = resultado + (e.curso -> (listaActual :+ e))
  }
  resultado
}
```

Implementación al estilo imperativo, parecido a como se haría en Java. En cada vuelta del bucle se recupera la lista de ese curso del Map (o una lista vacía si es la primera vez que aparece ese curso), se añade el estudiante con :+ y se reemplaza la entrada en el Map. Funciona correctamente, pero tiene más líneas de las necesarias y usa var para el Map acumulador, lo que introduce estado mutable.

```
def agruparPorCursoFuncional(es: List[Estudiante]): Map[String, List[Estud
iante]] =
  es.groupBy(_.curso)
```

La misma lógica en una línea. groupBy recorre la lista y agrupa los elementos según el valor que devuelva la función: aquí _.curso extrae el nombre del curso de cada estudiante y lo usa como clave del Map. Todo en un solo paso, sin estado intermedio. Dejé las dos versiones para poder comparar el resultado al final con == y verificar que son equivalentes.

```
val estudiantesArchivo: List[Estudiante] =
  Source.fromFile(rutaEntrada)
    .getLines()
    .filterNot(_.startsWith("#"))
    .filter(_.trim.nonEmpty)
    .flatMap(linea => parsearLinea(linea, separador))
    .toList
```

Lectura y preproceso del fichero en una cadena de operaciones. getLines() devuelve un iterador de líneas, y a partir de ahí se aplican filtros en orden: filterNot descarta los comentarios (líneas que empiezan por #), filter descarta las líneas vacías o que solo tengan espacios, y flatMap aplica parsearLinea a cada línea restante. flatMap sobre Option funciona como un filter + map: aplana los Some(estudiante) en la lista y descarta los None, así que las líneas que no se puedan parsear simplemente desaparecen sin necesidad de tratar el error explícitamente.

```
val filtrados = todosEstudiantes.filter(e => e.nota >= 7.0 && e.edad <= 2
2)
```

Filtrado con dos condiciones. Aquí no se puede usar el guión bajo abreviado porque el parámetro e se usa dos veces en la misma expresión, así que hay que darle nombre explícito. La condición combina nota mínima de 7 y edad máxima de 22, que en el contexto del programa representa estudiantes jóvenes con buen rendimiento.

```
val pw = new PrintWriter(new File(rutaSalida))
filtrados.foreach(e => pw.println(s"${e.id};${e.nombre};${e.edad};${e.not
a};${e.curso}"))
pw.close()
```

Escritura del resultado en disco. PrintWriter escribe línea a línea con println. El pw.close() al final es obligatorio: sin él, el buffer interno puede no llegar a vaciarse y el fichero quedará vacío o incompleto. El formato de salida es el mismo que el de entrada (campos separados por ;) para que el fichero filtrado pueda procesarse igual que el original.